

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«Класс MyString»

по дисциплине «Объектно-ориентированное программирование»

Выполнил
студент гр.5151001/40202

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Ерастов В.О.

Санкт-Петербург
2026г.

1. Цель работы.

Знакомство с базовыми понятиями объектно-ориентированного программирования «класс» и «объект» и принципом инкапсуляции.

2. Задачи

В рамках выполнения лабораторной работы были решены следующие задачи:

- реализовать класс для представления символьной строки на C++, не используя контейнеры и алгоритмы библиотеки STL;
- расширить функциональность интерпретатора Python с помощью реализованного класса.

Из дополнительных заданий методички были выполнены следующие:

- «Дополнительный функционал класса `MyString`»: `move`-конструктор и `move`-оператор присваивания, конструкторы, конвертирующие число (`int32_t`, `float`) в строку, операторы файлового ввода-вывода, поиск подстроки алгоритмом Ахо-Корасика, методы `at()`, `to_int()`, `to_float()`, а также собственный тип исключения для недопустимого преобразования;
- «Шаблон «Итератор»»: реализованы четыре типа итераторов (`iterator`, `const_iterator`, `reverse_iterator`, `const_reverse_iterator`) и версии методов, принимающие итератор вместо индекса.

3. Ход работы

3.1. Краткое описание класса `MyString` (формат хранения строки)

Класс `MyString` содержит в себе три поля: `pstr_` — указатель, для формата хранения символов была выбрана си-строка (динамический массив `char`, завершающийся нуль-терминатором `'\0'`); `len_` — длина строки от нулевого индекса до нуль-терминатора (без учёта завершающего символа); `capacity_` — размер выделенной под строку памяти, включая место под нуль-терминатор. Буфер выделяется оператором `new[]` и освобождается оператором `delete[]`, контейнеры и алгоритмы библиотеки STL для хранения строки не используются. Ёмкость увеличивается по мере необходимости при вставке и уменьшается только в методе `shrink_to_fit()`, что соответствует требованию методички. Копирование объекта — глубокое: выделяется новый буфер, в который копируются данные, поэтому копия не зависит от источника.

Так же в лабораторной работе для передачи строк разного типа (`std::string`, `const char*`, `MyString`) была использована структура `std::string_view` и был перегружен оператор неявного преобразования `operator std::string_view() const` для класса, чтобы `std::string_view` поддерживал класс `MyString` для передачи в методы класса. Благодаря этому каждая операция, принимающая строку-источник, реализована один раз, без дублирования кода под каждый из трёх типов. Все операции, работающие с индексами, проходят через единый приватный метод `check_index()`, который при выходе за границы строки генерирует исключение `std::out_of_range`.

3.2. Результирующий граф алгоритмической декомпозиции

Перед реализацией остальных методов было определено, какие функции являются базовыми (не выражаются через другие методы класса), а какие можно построить поверх них. Базовыми являются: `init()` — выделение буфера и копирование данных источника; `create_copy_of_pstr()` — создание копии текущего буфера нужного размера; `delete_pstr_change_params()` — освобождение старого буфера и

подстановка нового; `my_insert()` — единственная точка, в которой данные реально вставляются в буфер; `check_index()` — валидация индексов и количеств.

Все остальные методы выражаются через базовые: `append()` — это `insert()` в конец строки (`index = len_`), `replace()` — это `erase()` и `insert()`, `substr()` — копия объекта с последующим `erase()` лишних частей и `shrink_to_fit()`, `operator+` — копия объекта и `operator+=`, операторы сравнения построены на методе `compare()`, а `to_int()` — на `to_float()`.

Ниже на Рис. 1 представлен результирующий граф полной декомпозиции класса `MyString`.



Рис. 1. Граф алгоритмической декомпозиции

3.3. Результаты тестирования реализованных базовых методов

Тестирование базовых методов (конструкторы, `clear()`, `shrink_to_fit()`, операторы присваивания, геттеры) выполнялось программой `main.cpp`, повторяющей примеры использования из таблицы 1 методички: после каждой операции выводятся содержимое строки, её длина и ёмкость. Фрагмент вывода программы с пояснениями:

```
"Hello world!" (12, 13)  // конструктор от const char*
"hello" (5, 6)           // конструкторы от std::string и от
MyString
"hell" (4, 5)            // конструктор с ограничением по
количеству символов
"!!!!!" (5, 6)          // конструктор MyString(5, '!')
"" (0, 13)               // clear(): длина обнулена, ёмкость не
изменилась
"Hello!" (6, 13)         // erase(5, 6)
"Hello!" (6, 7)          // shrink_to_fit(): ёмкость стала равна
длине + 1
```

Геттеры: для строки «Hello world!» метод `size()` возвращает 12, `capacity()` — 13, `empty()` — 0; для строки, созданной конструктором по умолчанию, `empty()` возвращает 1. Операторы присваивания работают для источников типов `const char*`, `std::string`, `MyString` и `char`.

Проект поддерживает сборку компилятором clang++, в корне размещён файл `mystring_h_path.txt` с относительным путём к заголовочному файлу класса. Автоматическая тестовая система собирает тестовый исполняемый файл, прогоняет юнит-тесты всех методов класса и дополнительно проверяет отсутствие утечек памяти с помощью `valgrind`. Все юнит-тесты проходят, утечек памяти не обнаружено.

```

"123***4567890" (13, 1001)
"" (0, 0)
"Hello world!" (12, 13)
"hello" (5, 6)
"hello" (5, 6)
"hell" (4, 5)
"hell" (4, 5)
"hell" (4, 5)
"!!!!" (5, 6)
"Hello world!" (12, 13)
"" (0, 13)
"Hello world!" (12, 13)
"Hello!" (6, 13)
"Hello!" (6, 7)
"hello" (5, 6)
"hello std::string" (17, 18)
"hello MyString" (14, 18)
"!" (1, 2)
str
12
13
0
1
"!aaaaa" (6, 7)
"!aa@@aaa" (8, 9)
"a@@@@@aaaa" (10, 11)
"a@@@@@aaaa" (10, 11)
"a@@@@@aaaa" (10, 11)
"a@@@@@aaaa" (7, 11)
"abcaaaaa" (7, 11)
"!!!" (3, 4)
!!!
"!!!@@@" (6, 7)
"Hello!" (6, 13)
"hello wonderful world" (21, 22)
"hello wonder world" (18, 22)
"hello on world" (14, 22)
"amazing world" (13, 14)
"amazing" (7, 8)
"hel" (3, 4)
"lo" (2, 3)
"hello" (5, 6)
"hello" (5, 6)
"lo" (2, 3)
l
"heLlo" (5, 6)
-11
010011
6

```

Рис. 2. Проверка работы на примерах из методички

3.4. Примеры тестирования остальных методов

Для производных методов проверены все перегрузки: вставка символа и строки по индексу (в начало, середину и конец), добавление в конец, удаление, замена и извлечение подстроки. Фрагмент вывода программы с пояснениями:

```
"!aaaaa" (6, 7)           // insert(0, 1, '!')
"a@@@@@aaaa" (10, 11)    // insert(1, "@@@@@")
"abcaaaa" (7, 11)         // insert(1, "abcde", 1, 2)
"hello wonderful world" (21, 22) // replace(6, 7, "wonderful")
"hello on world" (14, 22) // replace(6, 7, "wonderful",
1, 2)
"amazing world" (13, 14)   // substr(6)
"amazing" (7, 8)          // substr(6, 7)
```

Операторы: `r = left + right` для строк «hel» и «lo» даёт «hello», при этом левый операнд не изменяется (создаётся временная копия), а `operator+=` изменяет исходный объект; `operator[]` позволяет как читать, так и изменять символ по индексу («hello» — `s[2] = 'L'` — «heLlo»). Сравнение: `compare()` для строк «abcd» и «abce» возвращает `-1`, операторы `==`, `!=`, `>`, `>=`, `<`, `<=` дают согласованные между собой результаты. Поиск: `find("amazing")` в строке «hello amazing world amazing» возвращает `6`, `find("amazing", 7)` — `20`, для ненайденной подстроки возвращается `-1`.

Отдельно проверено, что все методы, принимающие строку-источник (`insert`, `replace`, `find`, оператор присваивания), одинаково корректно работают с типами `const char*`, `std::string` и `MyString` — поддержка всех трёх типов `SOURCE_STR` подтверждена тестами.

3.5. Описание реализации Python-обёртки

Реализация Python обертки была выполнена с помощью библиотеки `pybind11`, класс `MyString` зарегистрирован как тип модуля `mystring`. В реализации была проблема с переносом структуры `std::string_view`: для библиотеки нет готового преобразования этого типа из Python-объектов, поэтому в обёртке реализована единая функция-конвертер `extract_string()`, которая строит `std::string` из Python-строки `str`, из объекта `bytes` и из другого объекта `MyString` (через `c_str()` и `size()`), после чего строка передаётся в C++-метод в виде `std::string_view`. Благодаря этому методы `insert()`,

`append()`, `replace()`, `find()` и операторы принимают все три требуемых типа источника одной и той же перегрузкой.

Множественные перегрузки конструкторов сведены в один диспетчер с необязательным вторым аргументом: он различает случаи `MyString("str")`, `MyString("str", count)`, `MyString(count, 'ch')`, `MyString(число)` и `MyString(другой MyString)` по типам переданных аргументов. Перегрузки `insert/append/replace/find/substr` реализованы лямбда-функциями с необязательными аргументами `py::arg`. Для остальных методов класса в обёртку переписаны объявления функций из заголовочного файла.

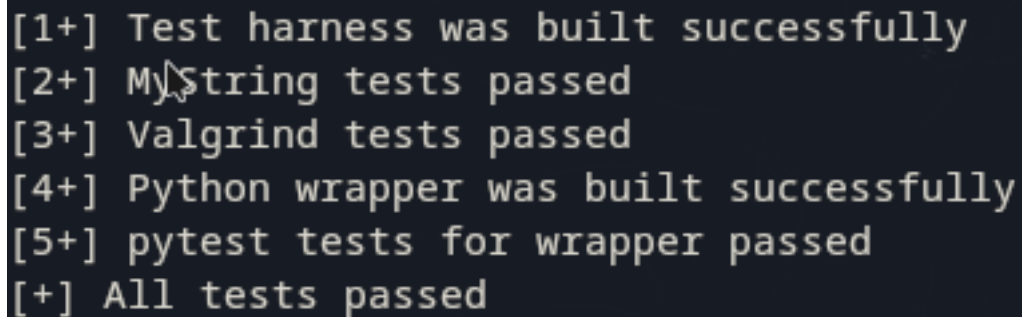
Для операций, которые нельзя экспортировать напрямую, написаны функции-«прослойки»: `assign()` и `assign_char()` — присваивание строки и символа (в Python оператор `=` не перегружается), `getitem()` и `setitem()` — реализация `__getitem__`/`__setitem__` поверх `operator[]`, `to_py_str()` — преобразование в строку Python для `__str__`/`__repr__`. Поддержаны стандартные протоколы Python: длина (`__len__`), строковое представление (`__str__`, `__repr__`), индексация (`__getitem__`, `__setitem__`), арифметика (`__add__`, `__iadd__`) и сравнение (`__eq__`, `__ne__`, `__lt__`, `__gt__`, `__le__`, `__ge__`) — объект `MyString` в Python внешне неотличим от обычной изменяемой строки.

3.6. Примеры тестирования Python-обёртки

Обёртка протестирована набором `pytest`-тестов: проверяются конструкторы из `str`, `bytes` и `MyString`, геттеры, вставка в начало, середину и конец строки, все варианты `append`, `erase` (в том числе то, что ёмкость при удалении не уменьшается), `clear()`, `replace()`, `substr()`, `find()` и операторы. Пример работы с объектом в интерпретаторе Python:

```
>>> from mystring import MyString
>>> s = MyString("Hello")
>>> s.append(" World")
>>> s.c_str()
'Hello World'
>>> s.find("World")
6
>>> s[0] = 'J'
>>> s.c_str()
'Jello World'
```

Полная цепочка автоматического тестирования (сборка проекта, юнит-тесты C++, проверка утечек памяти valgrind, сборка обёртки, pytest) завершается сообщением `[+] All tests passed` — все тесты проходят.

A screenshot of a terminal window with a dark background and light-colored text. It displays a sequence of six status messages, each on a new line. The messages are: '[1+] Test harness was built successfully', '[2+] MyString tests passed', '[3+] Valgrind tests passed', '[4+] Python wrapper was built successfully', '[5+] pytest tests for wrapper passed', and '[+] All tests passed'. A mouse cursor is visible over the second line.

```
[1+] Test harness was built successfully
[2+] MyString tests passed
[3+] Valgrind tests passed
[4+] Python wrapper was built successfully
[5+] pytest tests for wrapper passed
[+] All tests passed
```

Рис. 3. Результат тестирования python обертки

4. Ход работы по выполнению дополнительных заданий

4.1. Дополнительный функционал класса `MyString` (move-семантика, исключения)

В рамках задания реализованы move-конструктор и move-оператор присваивания: они забирают буфер у исходного объекта, а поля исходного объекта обнуляются, поэтому двойного освобождения памяти не происходит.

Сравнение move и сору-семантики: при копировании выделяется новый буфер и в него копируются все символы строки — стоимость $O(n)$ плюс аллокация памяти; при перемещении выполняется только присваивание трёх полей — $O(1)$, без аллокаций и копирования данных. Исходный объект после перемещения остаётся в валидном пустом состоянии, что подтверждено тестами: после `MyString moved(std::move(original))` новый объект содержит «MoveMe», а `original.size()` равен 0.

Также реализованы конструкторы `MyString(int32_t)` и `MyString(float)`, «собирающие» число посимвольными вставками через `insert()`; операторы файлового ввода-вывода `operator<<` и `operator>>`: первый записывает содержимое строки в файл, второй читает из файла очередное слово, пропуская пробельные символы (round-trip тест «запись с последующим чтением» пройден). Метод `find()` реализован алгоритмом Ахо-Корасика: по образцу строится префиксный автомат с суффиксными ссылками, и текст сканируется один раз, что даёт сложность $O(n + m)$ против $O(n \cdot m)$ у наивного поиска. Очередь обхода автомата реализована собственным классом `IntQueue`, так как контейнеры STL использовать нельзя. Перекрывающиеся вхождения образца обрабатываются корректно: `find("aa", 1)` в строке «aaaaaa» возвращает 1.

Примеры обработки исключений: при некорректных аргументах методы выбрасывают исключения стандартной библиотеки C++ — `at(100)` и вызовы `insert()/erase()/find()` с неверными индексами генерируют `std::out_of_range`, конструктор `MyString(-1, 'x')` — `std::invalid_argument`. Для недопустимого преобразования реализован собственный тип исключения `MyString::WrongTransformException` (на-

следник `std::exception`): вызов `to_float()` у строки, не являющейся числом, перехватывается, и программа продолжает работу:
`work exception to wrong transform`

4.2. Шаблон «Итератор»

Реализованы четыре типа итераторов: `iterator` — прямой итератор на основе «сырого» указателя `char*`, допускающий изменение строки; `const_iterator` — константный; `reverse_iterator` — реверсивный, у которого оператор инкремента сдвигает внутренний указатель к началу строки; `const_reverse_iterator` — константный реверсивный, наследник `reverse_iterator`. Для них определены методы `begin()/end()`, `cbegin()/cend()`, `rbegin()/rend()`, `rcbegin()/rcend()`. Для всех методов, принимающих индекс (`insert`, `erase`, `replace`, `substr`, `at`), реализованы версии с итератором: позиция вычисляется как разность итератора и `begin()` (или `cbegin()`).

Сравнение итератора и индекса: оба дают доступ к элементу за $O(1)$, так как итератор для данной структуры данных — тонкая обёртка над указателем. Индекс проще в использовании, но «привязан» к моменту времени: после вставки или удаления символов все индексы справа сдвигаются, и их нужно пересчитывать вручную. Итератор абстрагирует позицию от внутреннего хранения: один и тот же код обхода работает и с прямым, и с реверсивным обходом, а пара `begin()/end()` совместима с диапазонным `for` и алгоритмами стандартной библиотеки. Поэтому для обобщённого кода удобнее итератор, а индекс остаётся удобнее для точечного доступа по известной позиции.

Результаты тестирования итератора: обход строки «Hello» итераторами `begin()/end()` даёт «Hello»; модификация строки через `iterator` (перевод символов в верхний регистр) даёт «HELLO»; `const_iterator` и `const_reverse_iterator` позволяют только читать (попытка записи через них не компилируется); реверсивный обход даёт «olleH»; на пустой строке выполняется 0 итераций. Версии методов с итераторами дают ожидаемые результаты: `insert(it, 3, 'X')` в позицию 2 строки «Hello» даёт «HeXXXllo», `erase(it, 2)` из позиции 1 даёт «Hlo», `at(it)` возвращает

первый символ, `substr(it, 7)` с позиции 6 даёт «amazing». Все перечисленные проверки завершаются статусом [OK].

```
=== ТЕСТЫ ИТЕРАТОРОВ ===
begin()/end() обход: "Hello" (ожидали "Hello")
[OK] begin()/end()
После модификации: "HELLO" (ожидали "HELLO")
[OK] iterator modification
cbegin()/cend() обход: "World" (ожидали "World")
[OK] cbegin()/cend()
rbegin()/rend() обход: "olleH" (ожидали "olleH")
[OK] rbegin()/rend()
После модификации reverse: "ABC" (ожидали "ABC")
[OK] reverse_iterator modification
rcbegin()/rcend() обход: "olleH" (ожидали "olleH")
[OK] rcbegin()/rcend()
begin()/end() на пустой строке, итераций: 0 (ожидали 0)
[OK] empty string forward
insert(iterator, 3, 'X') в "Hello" на индекс 2: "HeXXXllo" (ожидали "HeXXXllo")
[OK] insert(iterator)
erase(iterator, 2) из "Hello" на индекс 1: "Hlo" (ожидали "Hlo")
[OK] erase(iterator)
at(begin()) = 'H' (ожидали 'H')
[OK] at(iterator)
substr(iterator) от индекса 6: "World" (ожидали "World")
[OK] substr(const_iterator)
```

Рис. 4. Результат тестирования итераторов

5. Выводы

В ходе лабораторной работы были изучены базовые понятия объектно-ориентированного программирования «класс» и «объект» и принцип инкапсуляции, реализован и протестирован класс `MyString`, а так же получены знания для расширения функциональности Python с помощью кода на C++. Дополнительно были выполнены задания повышенной сложности: освоены move-семантика и механизм исключений C++, реализован поведенческий паттерн «Итератор», а класс `MyString` расширен конвертирующими конструкторами, файловым вводом-выводом и поиском подстроки алгоритмом Ахо-Корасика.

Приложения

Приложение А

main.cpp

```
#include "my_string/my_string.h"
#include <string>
#include <iostream>
#include <cassert>
#include <fstream>
#include <cstdio>

void print_result(const std::string& test_name, bool passed)
{
    std::cout << (passed ? "[OK]   " : "[FAIL] ") << test_name << std::endl;
}

int main()
{
    MyString str_t1(1000, 'a');
    MyString str_t2("1234567890");
    str_t1.pstr();
    str_t1 = str_t2;
    str_t1.insert(3, "*****", 3);
    str_t1.pstr();

    MyString str;
    str.pstr();

    MyString str2("Hello world!");
    str2.pstr();

    std::string s_str = "hello";
    MyString str3(s_str);
    str3.pstr();

    MyString s_str1("hello");
    MyString str4(s_str1);
    str4.pstr();

    MyString str5("hello", 4);
    str5.pstr();

    MyString str6(std::string("hello"), 4);
    str6.pstr();

    MyString str7(MyString("hello"), 4);
```

```

str7.pstr();

MyString str8(5, '!');
str8.pstr();

MyString str_clear("Hello world!");
str_clear.pstr();
str_clear.clear();
str_clear.pstr();

MyString str_shrink("Hello world!");
str_shrink.pstr();
str_shrink.erase(5, 6);
str_shrink.pstr();
str_shrink.shrink_to_fit();
str_shrink.pstr();

MyString str_eq;
str_eq = "hello";
str_eq.pstr();
str_eq = std::string("hello std::string");
str_eq.pstr();
MyString str_eq_tmp("hello MyString");
str_eq = str_eq_tmp;
str_eq.pstr();

MyString str_eq_char;
str_eq_char = '!';
str_eq_char.pstr();

MyString str_cstr("str");
std::cout << str_cstr.c_str() << std::endl;

std::cout << str2.size() << std::endl;

std::cout << str2.capacity() << std::endl;

std::cout << str2.empty() << std::endl;
std::cout << str.empty() << std::endl;

MyString str_insert("aaaaa");
str_insert.insert(0, 1, '!');
str_insert.pstr();
str_insert.insert(3, 2, '@');
str_insert.pstr();

str_insert = "aaaaa";
str_insert.insert(1, "aaaaa");
str_insert.pstr();

```

```

str_insert = "aaaaa";
str_insert.insert(1, std::string("@@@@@"));
str_insert.pstr();

str_insert = "aaaaa";
MyString tmp("@@@@@");
str_insert.insert(1, tmp);
str_insert.pstr();

str_insert = "aaaaa";
str_insert.insert(1, "@@@@@", 2);
str_insert.pstr();

str_insert = "aaaaa";
str_insert.insert(1, "abcde", 1, 2);
str_insert.pstr();

MyString str_append;
str_append.append(3, '!');
str_append.pstr();
std::cout << str_append.c_str() << std::endl;
str_append.append(3, '@');
str_append.pstr();

str2 = "Hello world!";
str2.erase(5, 6);
str2.pstr();

MyString str_repl("hello amazing world");
str_repl.replace(6, 7, "wonderful");
str_repl.pstr();

str_repl = "hello amazing world";
str_repl.replace(6, 7, "wonderful", 6);
str_repl.pstr();

str_repl = "hello amazing world";
str_repl.replace(6, 7, "wonderful", 1, 2);
str_repl.pstr();

MyString str_sub("hello amazing world"), substr;
substr = str_sub.substr(6);
substr.pstr();

str_sub = "hello amazing world";
substr = str_sub.substr(6, 7);
substr.pstr();

MyString left("hel"), right("lo"), r;
r = left + right;

```

```

left.pstr();
right.pstr();
r.pstr();

left += right;
left.pstr();
right.pstr();

MyString str_brac("hello");
std::cout << str_brac[2] << std::endl;
str_brac[2] = 'L';
str_brac.pstr();

MyString a("abcd"), b("abce");
std::cout << a.compare(b) << b.compare(a) << std::endl;
std::cout << (a == b) << (a != b) << (a > b) << (a >= b) << (a < b)
    << (a <= b) << std::endl;

MyString str_find = std::string("hello");
str_find = "hello amazing world";
std::cout << str_find.find("amazing") << std::endl;

MyString str_num(0x12345678);
str_num.pstr();

MyString str_float(0.05f);
str_float.pstr();

str_num = std::move(str_float);
str_num.pstr();

std::cout << str_find.at(0) << std::endl;

MyString str_tofloat(0.05f);
MyString str_toint(12345678);
std::cout << str_tofloat.to_float() << std::endl;
std::cout << str_toint.to_int() << std::endl;

try
{
    std::cout << str_find.to_float() << std::endl;
}
catch (const MyString::WrongTransformException)
{
    std::cout << "work exception to wrong transform" << std::endl;
}

std::cout << "=== ТЕСТЫ МТЕПАТОРОБ ===" << std::endl;

// ----- 1-2. begin() / end() -----

```

```

{
    MyString s("Hello");
    std::string result;
    for (MyString::iterator it = s.begin(); it != s.end(); ++it)
        result += *it;

    std::cout << "begin()/end() обход: \"" << result
        << "\" (ожидали \"Hello\")" << std::endl;
    print_result("begin()/end()", result == "Hello");

    // Проверка модификации через iterator
    for (MyString::iterator it = s.begin(); it != s.end(); ++it)
        *it = std::toupper(*it);
    std::cout << "После модификации: \"" << s.c_str()
        << "\" (ожидали \"HELLO\")" << std::endl;
    print_result("iterator modification",
        std::string(s.c_str()) == "HELLO");
}

// ----- 3-4. cbegin() / cend() -----
{
    const MyString s("World");
    std::string result;
    for (MyString::const_iterator it = s.cbegin(); it != s.cend(); ++it)
        result += *it;

    std::cout << "cbegin()/cend() обход: \"" << result
        << "\" (ожидали \"World\")" << std::endl;
    print_result("cbegin()/cend()", result == "World");
    // *it = 'X'; // <-- должно НЕ компилироваться, если раскомментировать
}

// ----- 5-6. rbegin() / rend() -----
{
    MyString s("Hello");
    std::string result;
    for (MyString::reverse_iterator it = s.rbegin(); it != s.rend(); ++it)
        result += *it;

    std::cout << "rbegin()/rend() обход: \"" << result
        << "\" (ожидали \"olleH\")" << std::endl;
    print_result("rbegin()/rend()", result == "olleH");

    // Проверка модификации через reverse_iterator
    MyString s2("abc");
    for (MyString::reverse_iterator it = s2.rbegin(); it != s2.rend(); ++it)
        *it = std::toupper(*it);
    std::cout << "После модификации reverse: \"" << s2.c_str()
        << "\" (ожидали \"ABC\")" << std::endl;
    print_result("reverse_iterator modification",

```

```

        std::string(s2.c_str()) == "ABC");
    }

// ----- 7-8. rcbegin() / rcend() -----
{
    const MyString s("Hello");
    std::string result;
    for (MyString::const_reverse_iterator it = s.rcbegin(); it != s.rcend();
        ++it)
        result += *it;

    std::cout << "rcbegin()/rcend() обход: \"" << result
        << "\" (ожидали \"olleH\")" << std::endl;
    print_result("rcbegin()/rcend()", result == "olleH");
    // *it = 'X'; // <-- должно НЕ компилироваться, если раскомментировать
}

// ----- Проверка на пустой строке (граничный случай) -----
{
    MyString s;
    int count = 0;
    for (MyString::iterator it = s.begin(); it != s.end(); ++it)
        ++count;
    std::cout << "begin()/end() на пустой строке, итераций: " << count
        << " (ожидали 0)" << std::endl;
    print_result("empty string forward", count == 0);
}

// ----- insert/erase/replace/at через iterator -----
{
    MyString s("Hello");
    MyString::iterator it = s.begin();
    ++it;
    ++it; // указывает на второй 'l' (индекс 2)
    s.insert(it, 3, 'X');
    std::cout << "insert(iterator, 3, 'X') в \"Hello\" на индекс 2: \""
        << s.c_str() << "\" (ожидали \"HeXXXllo\")" << std::endl;
    print_result("insert(iterator)", std::string(s.c_str()) == "HeXXXllo");
}

{
    MyString s("Hello");
    MyString::iterator it = s.begin();
    ++it; // индекс 1 ('e')
    s.erase(it, 2);
    std::cout << "erase(iterator, 2) из \"Hello\" на индекс 1: \""
        << s.c_str() << "\" (ожидали \"Hlo\")" << std::endl;
    print_result("erase(iterator)", std::string(s.c_str()) == "Hlo");
}

{
    MyString s("Hello");

```

```

MyString::iterator it = s.begin();
char c = s.at(it);
std::cout << "at(begin()) = '" << c << "' (ждали 'H') " << std::endl;
print_result("at(iterator)", c == 'H');
}
{
MyString s("Hello World");
MyString::const_iterator it = s.cbegin();
for (int i = 0; i < 6; ++i)
    ++it; // индекс 6 -> "World"
MyString sub = s.substr(it);
std::cout << "substr(iterator) от индекса 6: \"" << sub.c_str()
    << "\" (ждали \"World\") " << std::endl;
print_result("substr(const_iterator)",
    std::string(sub.c_str()) == "World");
}

std::cout << std::endl << "=== ОСТАЛЬНЫЕ ФУНКЦИИ ===" << std::endl;

// ----- MyString(MyString&&) - move-конструктор -----
{
MyString original("MoveMe");
const char* original_ptr = original.c_str();

MyString moved(std::move(original));

std::cout << "Move-конструктор: moved = \"" << moved.c_str()
    << "\" (ждали \"MoveMe\") " << std::endl;
print_result("move constructor value",
    std::string(moved.c_str()) == "MoveMe");

std::cout << "После move, original.size() = " << original.size()
    << " (ждали 0) " << std::endl;
print_result("move constructor leaves source empty",
    original.size() == 0);
}

// ----- MyString(int32_t) - число в строку -----
{
MyString s(12345);
std::cout << "MyString(12345) = \"" << s.c_str()
    << "\" (ждали \"12345\") " << std::endl;
print_result("int constructor positive",
    std::string(s.c_str()) == "12345");

MyString s_neg(-42);
std::cout << "MyString(-42) = \"" << s_neg.c_str()
    << "\" (ждали \"-42\") " << std::endl;
print_result("int constructor negative",
    std::string(s_neg.c_str()) == "-42");
}

```

```

}

// ----- MyString(float) – float в строку -----
{
    MyString s(3.14f);
    std::cout << "MyString(3.14f) = \"" << s.c_str()
        << "\" (ожидали что-то вроде \"3.14...\")" << std::endl;
    // Точное сравнение зависит от реализации округления, поэтому просто
    // выводим
}
{
    MyString s(0.05f);
    std::cout << "MyString(0.05f) = \"" << s.c_str()
        << "\" (ожидали что-то вроде \"0.05...\")" << std::endl;
}

// ----- operator=(MyString&&) – move-присваивание -----
{
    MyString a("First");
    MyString b("Second");
    b = std::move(a);

    std::cout << "После move-присваивания b = \"" << b.c_str()
        << "\" (ожидали \"First\")" << std::endl;
    print_result("move assignment value",
        std::string(b.c_str()) == "First");

    std::cout << "После move-присваивания a.size() = " << a.size()
        << " (ожидали 0)" << std::endl;
    print_result("move assignment leaves source empty", a.size() == 0);
}

// ----- at(index) -----
{
    MyString s("Test");
    char c = s.at(1);
    std::cout << "at(1) для \"Test\" = '" << c << "' (ожидали 'e')"
        << std::endl;
    print_result("at(index) valid", c == 'e');

    bool threw = false;
    try
    {
        s.at(100); // индекс за пределами строки
    }
    catch (const std::exception&)
    {
        threw = true;
    }
    std::cout << "at(100) выбросил исключение: " << (threw ? "да" : "нет")

```

```

        << " (ждали да)" << std::endl;
    print_result("at(index) throws on invalid index", threw);
}

// ----- to_int() -----
{
    MyString s("123");
    int result = s.to_int();
    std::cout << "to_int() для \"123\" = " << result << " (ждали 123)"
        << std::endl;
    print_result("to_int() positive", result == 123);
}

// ----- to_float() -----
{
    MyString s("3.14");
    float result = s.to_float();
    std::cout << "to_float() для \"3.14\" = " << result
        << " (ждали ~3.14)" << std::endl;
    print_result("to_float()", std::abs(result - 3.14f) < 0.01f);
}

std::cout << std::endl
    << "=== ТЕСТЫ operator<< / operator>> ===" << std::endl;

const char* io_test_file = "mystring_io_test.txt";

// ----- operator<<(ofstream, MyString) -----
{
    MyString to_write("Hello, file!");
    {
        std::ofstream out(io_test_file);
        out << to_write;
    }

    std::ifstream check(io_test_file);
    std::string raw((std::istreambuf_iterator<char>(check)),
        std::istreambuf_iterator<char>());

    std::cout << "operator<< записал в файл: \"" << raw
        << "\" (ждали \"Hello, file!\")" << std::endl;
    print_result("operator<<(ofstream, MyString)", raw == "Hello, file!");
}

// ----- operator>>(ifstream, MyString): чтение нескольких "слов"
// -----
{
    {
        std::ofstream out(io_test_file);
        out << "    first    second\\tthird ";
    }
}

```

```

}

std::ifstream in(io_test_file);
MyString w1, w2, w3;
in >> w1;
in >> w2;
in >> w3;

std::cout << "operator>> прочитал: \"" << w1.c_str() << "\", \""
    << w2.c_str() << "\", \"" << w3.c_str() << "\"" << std::endl;
print_result(
    "operator>>(ifstream, MyString) #1 (пропуск пробелов в начале)",
    std::string(w1.c_str()) == "first");
print_result(
    "operator>>(ifstream, MyString) #2 (несколько слов подряд)",
    std::string(w2.c_str()) == "second");
print_result("operator>>(ifstream, MyString) #3 (разделитель - таб)",
    std::string(w3.c_str()) == "third");
}

// ----- operator>>: конец файла посреди чтения -----
{
    {
        std::ofstream out(io_test_file);
        out << "onlyword";
    }

    std::ifstream in(io_test_file);
    MyString w;
    in >> w;
    std::cout << "operator>> без пробела до EOF: \"" << w.c_str()
        << "\" (ожидали \"onlyword\")" << std::endl;
    print_result("operator>> дочитывает слово до EOF",
        std::string(w.c_str()) == "onlyword");
}

// ----- Круговой прогон: operator<< затем operator>> -----
{
    MyString original("roundtrip");
    {
        std::ofstream out(io_test_file);
        out << original;
    }

    MyString restored;
    std::ifstream in(io_test_file);
    in >> restored;

    std::cout << "round-trip << / >>: \"" << restored.c_str()
        << "\" (ожидали \"roundtrip\")" << std::endl;
}

```

```

        print_result("operator<< / operator>> round-trip",
            restored == original);
    }

    std::remove(io_test_file);

    std::cout << std::endl
        << "=== ТЕСТЫ find() (алгоритм Ахо-Корасика) ===" << std::endl;

    // ----- Базовые случаи (уже проверялись выше, но проверим явно)
    // -----
    {
        MyString s("hello amazing world amazing");
        print_result("find: первое вхождение", s.find("amazing") == 6);
        print_result("find: поиск с индекса", s.find("amazing", 7) == 20);
        print_result("find: образец не найден", s.find("notfound") == -1);
        print_result("find: пустой образец возвращает 0", s.find("") == 0);
    }

    // ----- Перекрывающиеся вхождения: ключевая проверка для
    // суффиксных ссылок автомата Ахо-Корасика -----
    {
        MyString s("aaaaaa");
        print_result("find: перекрытие 'aa' от начала", s.find("aa") == 0);
        print_result("find: перекрытие 'aa' от индекса 1",
            s.find("aa", 1) == 1);
        print_result("find: перекрытие 'aa' от индекса 5",
            s.find("aa", 5) == -1);

        MyString s2("abababab");
        print_result("find: перекрытие 'abab' от 0", s2.find("abab") == 0);
        print_result("find: перекрытие 'abab' от 1", s2.find("abab", 1) == 2);
    }

    // ----- Образец в начале / в конце / из одного символа -----
    {
        MyString s("abcdef");
        print_result("find: образец в начале строки", s.find("abc") == 0);
        print_result("find: образец в конце строки", s.find("def") == 3);
        print_result("find: образец из одного символа", s.find("c") == 2);
    }

    // ----- Образец длиннее строки / равен строке целиком -----
    {
        MyString s("abc");
        print_result("find: образец длиннее строки", s.find("abcdef") == -1);

        MyString s_exact("exact");
        print_result("find: образец равен всей строке",
            s_exact.find("exact") == 0);
    }

```

```

}

// ----- Некорректный индекс должен бросать исключение -----
{
    MyString s("hello");
    bool threw = false;
    try
    {
        s.find("l", 100);
    }
    catch (const std::exception&)
    {
        threw = true;
    }
    std::cout << "find с индексом за пределами строки выбросил "
                "исключение: "
                << (threw ? "да" : "нет") << " (ожидали да)" << std::endl;
    print_result("find: исключение при некорректном индексе", threw);
}

std::cout << std::endl
            << "=== ТЕСТЫ append(SOURCE_STR, ...) ===" << std::endl;

// ----- append(SOURCE_STR) -----
{
    MyString s;
    s.append(std::string_view("Hello "));
    s.append(std::string_view("world"));
    std::cout << "append(string_view) x2: \"" << s.c_str()
                << "\" (ожидали \"Hello world\")" << std::endl;
    print_result("append(SOURCE_STR)",
                std::string(s.c_str()) == "Hello world");
}

// ----- append(SOURCE_STR, int count) -----
{
    MyString s;
    s.append(std::string_view("Hello world"), 6);
    std::cout << "append(string_view, 6): \"" << s.c_str()
                << "\" (ожидали \"Hello \")" << std::endl;
    print_result("append(SOURCE_STR, count)",
                std::string(s.c_str()) == "Hello ");
}

// ----- append(SOURCE_STR, int s_index, int count) -----
{
    MyString s;
    s.append(std::string_view("Hello world"), 6, 5);
    std::cout << "append(string_view, 6, 5): \"" << s.c_str()
                << "\" (ожидали \"world\")" << std::endl;
}

```

```

    print_result("append(SOURCE_STR, s_index, count)",
        std::string(s.c_str()) == "world");
}

std::cout << std::endl
    << "=== ТЕСТЫ insert(iterator, SOURCE_STR, ...) ===" << std::endl;

// ----- insert(iterator, SOURCE_STR) -----
{
    MyString s("aaaaa");
    MyString::iterator it = s.begin();
    ++it; // индекс 1
    s.insert(it, "@@@@");
    std::cout << "insert(iterator, \@@@@\") на индекс 1: \"" << s.c_str()
        << "\" (ожидали \@@@@aaaa)\")" << std::endl;
    print_result("insert(iterator, SOURCE_STR)",
        std::string(s.c_str()) == "a@@@@aaaa");
}

// ----- insert(iterator, SOURCE_STR, int count) -----
{
    MyString s("aaaaa");
    MyString::iterator it = s.begin();
    ++it; // индекс 1
    s.insert(it, "@@@@", 2);
    std::cout << "insert(iterator, \@@@@\ ", 2) на индекс 1: \""
        << s.c_str() << "\" (ожидали \@@@@aaaa)\")" << std::endl;
    print_result("insert(iterator, SOURCE_STR, count)",
        std::string(s.c_str()) == "a@@@@aaaa");
}

// ----- insert(iterator, SOURCE_STR, int s_index, int count)
// -----
{
    MyString s("aaaaa");
    MyString::iterator it = s.begin();
    ++it; // индекс 1
    s.insert(it, "abcde", 1, 2);
    std::cout << "insert(iterator, \abcde\ ", 1, 2) на индекс 1: \""
        << s.c_str() << "\" (ожидали \abcaaaaa)\")" << std::endl;
    print_result("insert(iterator, SOURCE_STR, s_index, count)",
        std::string(s.c_str()) == "abcaaaaa");
}

std::cout << std::endl
    << "=== ТЕСТЫ replace(iterator, ...) ===" << std::endl;

// ----- replace(iterator, count, SOURCE_STR) -----
{
    MyString s("hello amazing world");

```

```

MyString::iterator it = s.begin();
for (int i = 0; i < 6; ++i)
    ++it; // индекс 6

s.replace(it, 7, "wonderful");
std::cout << "replace(iterator@6, 7, \"wonderful\"): \"" << s.c_str()
    << "\" (ожидали \"hello wonderful world\")" << std::endl;
print_result("replace(iterator, count, SOURCE_STR)",
    std::string(s.c_str()) == "hello wonderful world");
}

// ----- replace(iterator, count, SOURCE_STR, s_count) -----
{
    MyString s("hello amazing world");
    MyString::iterator it = s.begin();
    for (int i = 0; i < 6; ++i)
        ++it;

    s.replace(it, 7, "wonderful", 6);
    std::cout << "replace(iterator@6, 7, \"wonderful\", 6): \"" << s.c_str()
        << "\" (ожидали \"hello wonder world\")" << std::endl;
    print_result("replace(iterator, count, SOURCE_STR, s_count)",
        std::string(s.c_str()) == "hello wonder world");
}

// ----- replace(iterator, count, SOURCE_STR, s_index, s_count)
// -----
{
    MyString s("hello amazing world");
    MyString::iterator it = s.begin();
    for (int i = 0; i < 6; ++i)
        ++it;

    s.replace(it, 7, "wonderful", 1, 2);
    std::cout << "replace(iterator@6, 7, \"wonderful\", 1, 2): \""
        << s.c_str() << "\" (ожидали \"hello on world\")"
        << std::endl;
    print_result("replace(iterator, count, SOURCE_STR, s_index, s_count)",
        std::string(s.c_str()) == "hello on world");
}

std::cout << std::endl
    << "=== ТЕСТЫ substr(const_iterator, count) ===" << std::endl;
{
    MyString s("hello amazing world");
    MyString::const_iterator it = s.cbegin();
    for (int i = 0; i < 6; ++i)
        ++it; // индекс 6

    MyString sub = s.substr(it, 7);

```

```

        std::cout << "substr(const_iterator@6, 7): \"" << sub.c_str()
            << "\" (ожидали \"amazing\")" << std::endl;
        print_result("substr(const_iterator, count)",
            std::string(sub.c_str()) == "amazing");
    }

    std::cout << std::endl << "=== ТЕСТЫ самоприсваивания ===" << std::endl;

    // ----- operator=(const char*) при самоприсваивании -----
    {
        MyString s("self-assign test");
        s = s.c_str();
        std::cout << "self-assign через c_str(): \"" << s.c_str()
            << "\" (ожидали \"self-assign test\")" << std::endl;
        print_result("operator=(const char*) self-assignment",
            std::string(s.c_str()) == "self-assign test");
    }

    // ----- operator=(const MyString&) при самоприсваивании -----
    {
        MyString s("self-assign myself");
        s = s;
        std::cout << "self-assign через operator=(const MyString&): \""
            << s.c_str() << "\" (ожидали \"self-assign myself\")"
            << std::endl;
        print_result("operator=(const MyString&) self-assignment",
            std::string(s.c_str()) == "self-assign myself");
    }

    std::cout << std::endl << "=== ТЕСТЫ ЗАВЕРШЕНЫ ===" << std::endl;
    return 0;
}

```

my_string.cpp

```

#include "my_string.h"
#include <cstring>
#include <stdexcept>
#include <string_view>

#ifdef DEBUG
#include <iostream>
#endif

class AhoCorasickNode
{
public:
    static const int kAlphabetSize = 256;

    AhoCorasickNode()
    {

```

```

    for (int ch = 0; ch < kAlphabetSize; ++ch)
    {
        children_[ch] = -1;
        next_[ch] = -1;
    }
}

int child(int ch) const { return children_[ch]; }
void set_child(int ch, int node) { children_[ch] = node; }

int cached_transition(int ch) const { return next_[ch]; }
void cache_transition(int ch, int node) { next_[ch] = node; }

int fail() const { return fail_; }
void set_fail(int node) { fail_ = node; }

bool is_terminal() const { return is_terminal_; }
void mark_terminal() { is_terminal_ = true; }

private:
int children_[kAlphabetSize];
int next_[kAlphabetSize];
int fail_ = 0;
bool is_terminal_ = false;
};

class IntQueue
{
public:
    explicit IntQueue(int capacity) { data_ = new int[capacity]; }

    ~IntQueue() { delete[] data_; }

    void push(int value) { data_[tail_++] = value; }
    int pop() { return data_[head_++]; }
    bool empty() const { return head_ == tail_; }

private:
int* data_;
int head_ = 0;
int tail_ = 0;
};

class AhoCorasickAutomaton
{
public:
    AhoCorasickAutomaton(const char* pattern, int pattern_len)
    {
        capacity_ = pattern_len + 1;
        nodes_ = new AhoCorasickNode*[capacity_];
    }
};

```

```

    add_node();

    insert_pattern(pattern, pattern_len);
    build_fail_links();
}

~AhoCorasickAutomaton()
{
    for (int i = 0; i < size_; ++i)
        delete nodes_[i];
    delete[] nodes_;
}

AhoCorasickAutomaton(const AhoCorasickAutomaton&) = delete;
AhoCorasickAutomaton& operator=(const AhoCorasickAutomaton&) = delete;

void step(unsigned char ch) { state_ = go(state_, ch); }

bool is_match() const { return nodes_[state_]->is_terminal(); }

private:
AhoCorasickNode** nodes_;
int size_ = 0;
int capacity_;
int state_ = 0;

int add_node()
{
    nodes_[size_] = new AhoCorasickNode();
    return size_++;
}

void insert_pattern(const char* pattern, int pattern_len)
{
    int current = 0;

    for (int i = 0; i < pattern_len; ++i)
    {
        unsigned char ch = static_cast<unsigned char>(pattern[i]);

        if (nodes_[current]->child(ch) == -1)
            nodes_[current]->set_child(ch, add_node());

        current = nodes_[current]->child(ch);
    }

    nodes_[current]->mark_terminal();
}

int go(int state, unsigned char ch)

```

```

{
    if (nodes_[state]->cached_transition(ch) != -1)
        return nodes_[state]->cached_transition(ch);

    int result;
    if (nodes_[state]->child(ch) != -1)
        result = nodes_[state]->child(ch);
    else if (state == 0)
        result = 0;
    else
        result = go(nodes_[state]->fail(), ch);

    nodes_[state]->cache_transition(ch, result);
    return result;
}

void build_fail_links()
{
    IntQueue queue(capacity_);

    for (int ch = 0; ch < AhoCorasickNode::kAlphabetSize; ++ch)
    {
        int child = nodes_[0]->child(ch);
        if (child == -1)
            continue;

        nodes_[child]->set_fail(0);
        queue.push(child);
    }

    while (!queue.empty())
    {
        int current = queue.pop();

        for (int ch = 0; ch < AhoCorasickNode::kAlphabetSize; ++ch)
        {
            int child = nodes_[current]->child(ch);
            if (child == -1)
                continue;

            int fail_state = go(nodes_[current]->fail(), ch);
            nodes_[child]->set_fail(fail_state);

            if (nodes_[fail_state]->is_terminal())
                nodes_[child]->mark_terminal();

            queue.push(child);
        }
    }
}

```

```

};

void MyString::init(std::string_view sv)
{
    len_ = sv.size();
    capacity_ = len_ + 1;
    pstr_ = new char[capacity_];
    std::memcpy(pstr_, sv.data(), len_);
    pstr_[len_] = '\0';
}

char* MyString::create_copy_of_pstr(int new_size) const
{
    char* tmp = new char[new_size];
    if (new_size < capacity_)
        std::memcpy(tmp, pstr_, new_size);
    else
        std::memcpy(tmp, pstr_, capacity_);

    return tmp;
}

void MyString::delete_pstr_change_params(char* new_pstr, int new_len,
                                         int new_capacity)
{
    delete[] pstr_;
    len_ = new_len;
    capacity_ = new_capacity;
    pstr_ = new_pstr;
}

void MyString::my_insert(int index, int count, const char* data)
{
    if (capacity_ == 0)
    {
        pstr_ = new char[1];
        pstr_[0] = '\0';
        len_ = 0;
        capacity_ = 1;
    }

    index = check_index(index, len_, 0, "Insert or append");

    int new_capacity = len_ + count + 1;
    if (len_ + count < capacity_)
        new_capacity = capacity_;

    char* tmp = create_copy_of_pstr(new_capacity);
    std::memmove(tmp + index + count, tmp + index, len_ + 1 - index);
    std::memcpy(tmp + index, data, count);
}

```

```

    delete_pstr_change_params(tmp, len_ + count, new_capacity);
}

int MyString::check_index(int index, int capacity, int count,
                          std::string error_msg) const
{
    if (index > capacity || index < 0)
        throw std::out_of_range(
            "Index is bigger than length of string. \n Error in method " +
            error_msg + "\n");
    if (count < 0 || index + count > capacity)
        throw std::out_of_range("Count is not right in method " + error_msg +
            "\n");

    return index;
}

#ifdef DEBUG
void MyString::pstr()
{
    std::cout << "\n";
    for (int i = 0; i < len_; ++i)
    {
        std::cout << pstr_[i];
    }
    std::cout << "\n";
    std::cout << " " << "(" << len_ << ", " << capacity_ << ")" << std::endl;
}
#endif

MyString::iterator MyString::begin()
{
    iterator tmp(pstr_);
    return tmp;
}

MyString::iterator MyString::end()
{
    iterator tmp(pstr_ + len_);
    return tmp;
}

MyString::const_iterator MyString::cbegin() const
{
    const_iterator tmp(pstr_);
    return tmp;
}

MyString::const_iterator MyString::cend() const
{
    const_iterator tmp(pstr_ + len_);
    return tmp;
}

```

```

}
MyString::reverse_iterator MyString::rbegin()
{
    reverse_iterator tmp(pstr_ + len_ - 1);
    return tmp;
}
MyString::reverse_iterator MyString::rend()
{
    reverse_iterator tmp(pstr_ - 1);
    return tmp;
}
MyString::const_reverse_iterator MyString::rcbegin() const
{
    const_reverse_iterator tmp(pstr_ + len_ - 1);
    return tmp;
}
MyString::const_reverse_iterator MyString::rcend() const
{
    const_reverse_iterator tmp(pstr_ - 1);
    return tmp;
}

MyString::MyString() : len_(0), capacity_(0) {}

MyString::MyString(std::string_view source_str) { init(source_str); }

MyString::MyString(std::string_view source_str, int count)
{
    init(source_str.substr(0, count)); // Create substring of original string
    // that starts at 0 and ends in count
}

MyString::MyString(int count, char ch)
{
    if (count < 0)
        throw std::invalid_argument(
            "Count is must be more than zero/ Error in constructor");
    len_ = count;
    capacity_ = len_ + 1;
    pstr_ = new char[capacity_];

    std::memset(pstr_, ch, count);
    pstr_[capacity_ - 1] = '\0';
}

MyString::MyString(const MyString& other) { init(std::string_view(other)); }

MyString::MyString(const char* source_str)
    : MyString(std::string_view(source_str))
{

```

```

}

MyString::MyString(std::string source_str)
    : MyString(std::string_view(source_str))
{
}

MyString::MyString(MyString&& other)
    : pstr_(other.pstr_), len_(other.len_), capacity_(other.capacity_)
{
    other.pstr_ = nullptr;
    other.len_ = 0;
    other.capacity_ = 0;
}

MyString::MyString(int32_t number)
{
    if (number == 0)
    {
        this->insert(0, 1, '0');
    }

    bool is_negative = false;
    int32_t tmp = number;

    if (tmp < 0)
    {
        is_negative = true;
        tmp = -tmp;
    }

    while (tmp > 0)
    {
        char tmp2 = '0' + tmp % 10;
        this->insert(0, 1, tmp2);
        tmp /= 10;
    }

    if (is_negative)
        this->insert(0, 1, '-');
}

MyString::MyString(float number)
{
    if (number < 0)
    {
        this->insert(0, 1, '-');
    }

    int32_t int_part = static_cast<int32_t>(number);

```

```

MyString int_str(int_part);

for (int i = 0; i < int_str.size(); ++i)
{
    this->insert(this->size(), 1, int_str[i]);
}

float fraction = number - int_part;

if (fraction > 0.00001f)
{
    this->insert(this->size(), 1, '.');

    int steps = 0;

    while (steps < 6)
    {
        fraction *= 10.0f;
        int32_t digit = static_cast<int32_t>(fraction);

        char ch = '0' + digit;
        this->insert(this->size(), 1, ch);

        fraction -= digit;
        ++steps;

        if (fraction < 0.00001f)
        {
            break;
        }
    }
}

MyString::~MyString() { delete[] pstr_; }

void MyString::clear()
{
    std::memset(pstr_, 0, capacity_);
    len_ = 0;
}

void MyString::shrink_to_fit()
{
    char* tmp = create_copy_of_pstr(len_ + 1);
    delete_pstr_change_params(tmp, len_, len_ + 1);
}

void MyString::operator=(std::string_view source_str)
{

```

```

int new_len = static_cast<int>(source_str.size());

if (new_len + 1 <= capacity_)
{
    std::memcpy(pstr_, source_str.data(), new_len);
    pstr_[new_len] = '\0';
    len_ = new_len;
}
else
{
    delete_pstr_change_params(nullptr, 0, 0);
    init(source_str);
}
}

void MyString::operator=(const char* source_str)
{
    if (source_str != pstr_)
    {
        *this = std::string_view(source_str);
    }
}

void MyString::operator=(const std::string& source_str)
{
    *this = std::string_view(source_str);
}

void MyString::operator=(char ch)
{
    if (2 <= capacity_)
    {
        pstr_[0] = ch;
        pstr_[1] = '\0';
        len_ = 1;
    }
    else
    {
        delete_pstr_change_params(new char[2], 1, 2);
        pstr_[0] = ch;
        pstr_[1] = '\0';
    }
}

MyString& MyString::operator=(MyString&& other)
{
    if (this != &other)
    {
        delete_pstr_change_params(nullptr, 0, 0);
    }
}

```

```

    pstr_ = other.pstr_;
    len_ = other.len_;
    capacity_ = other.capacity_;

    other.pstr_ = nullptr;
    other.len_ = 0;
    other.capacity_ = 0;
}

return *this;
}

MyString& MyString::operator=(const MyString& other)
{
    if (this != &other)
    {
        *this = std::string_view(other);
    }
    return *this;
}

const char* MyString::c_str() const { return pstr_; }

int MyString::size() const { return len_; }

int MyString::capacity() const { return capacity_; }

bool MyString::empty() const { return len_ == 0; }

void MyString::insert(int index, int count, char ch)
{
    if (count < 0)
        throw std::out_of_range("Count is negative in method insert by char");

    char* data = new char[count];

    std::memset(data, ch, count);
    my_insert(index, count, data);

    delete[] data;
}

void MyString::insert(int index, std::string_view source_str)
{
    my_insert(index, source_str.size(), source_str.data());
}

void MyString::insert(int index, std::string_view source_str, int count)
{
    my_insert(index, count, source_str.data());
}

```

```

}

void MyString::insert(int index, std::string_view source_str, int s_index,
                    int count)
{
    std::string_view new_source_str = source_str.substr(s_index, count);
    my_insert(index, count, new_source_str.data());
}

void MyString::insert(iterator it, int count, char ch)
{
    int index = it - begin();
    insert(index, count, ch);
}

void MyString::insert(iterator it, std::string_view source_str)
{
    insert(it, source_str, 0, source_str.size());
}

void MyString::insert(iterator it, std::string_view source_str, int count)
{
    insert(it, source_str, 0, count);
}

void MyString::insert(iterator it, std::string_view source_str, int s_index,
                    int count)
{
    int index = it - begin();
    insert(index, source_str, s_index, count);
}

void MyString::append(int count, char ch) { insert(len_, count, ch); }

void MyString::append(std::string_view source_str) { insert(len_, source_str); }

void MyString::append(std::string_view source_str, int count)
{
    insert(len_, source_str, count);
}

void MyString::append(std::string_view source_str, int s_index, int count)
{
    insert(len_, source_str, s_index, count);
}

void MyString::erase(int index, int count)
{
    index = check_index(index, len_, count, "erase");

```

```

std::memset(pstr_ + index, 0, count);
std::memmove(
    pstr_ + index, pstr_ + index + count,
    capacity_ - index -
    count); // Move the other part of string that didnt erase to start.
            // dest is on index where start to erase, src is the place
            // where erase ends, count is whole len - part that erased
            // and where index starts
len_ = len_ - count;
}

void MyString::erase(iterator it, int count)
{
    int index = it - begin();
    erase(index, count);
}

void MyString::replace(int index, int count, std::string_view source_str)
{
    replace(index, count, source_str, 0, source_str.size());
}

void MyString::replace(int index, int count, std::string_view source_str,
                        int s_count)
{
    replace(index, count, source_str, 0, s_count);
}

void MyString::replace(int index, int count, std::string_view source_str,
                        int s_index, int s_count)
{
    std::string_view replace_str = source_str.substr(s_index, s_count);
    erase(index, count);
    insert(index, replace_str);
}

void MyString::replace(iterator it, int count, std::string_view source_str)
{
    replace(it, count, source_str, 0, source_str.size());
}

void MyString::replace(iterator it, int count, std::string_view source_str,
                        int s_count)
{
    replace(it, count, source_str, 0, s_count);
}

void MyString::replace(iterator it, int count, std::string_view source_str,
                        int s_index, int s_count)
{

```

```

    int index = it - begin();
    replace(index, count, source_str, s_index, s_count);
}

MyString MyString::substr(int index) const
{
    return substr(index, len_ - index);
}

MyString MyString::substr(int index, int count) const
{
    index = check_index(index, len_, count, "substr");

    MyString new_str = *this; // Unname pointer and make copy

    if (index != 0)
        new_str.erase(0, index); // delete start of the string previous to
                                // index were we need substr

    new_str.erase(count, len_ - index -
                  count); // delete other part after substr to end

    new_str.shrink_to_fit();

    return new_str;
}

MyString MyString::substr(const_iterator it) const
{
    int index = it - cbegin();
    return substr(it, len_ - index);
}

MyString MyString::substr(const_iterator it, int count) const
{
    int index = it - cbegin();
    return substr(index, count);
}

MyString& MyString::operator+=(std::string_view source_str)
{
    this->append(source_str);
    return *this;
}

MyString MyString::operator+(std::string_view source_str) const
{
    MyString tmp = *this;
    return tmp += source_str;
}

```

```

char& MyString::operator[](int index)
{
    index = check_index(index, len_, 0, "operator [] changing index");

    return pstr_[index];
}

const char& MyString::operator[](int index) const
{
    index = check_index(index, len_, 0, "operator [] getting index");
    return pstr_[index];
}

short MyString::compare(const MyString& other) const
{
    int this_len = this->len_;

    for (int i = 0; i < this_len; ++i)
    {
        if (other.len_ < i)
            return 1;

        if (this->pstr_[i] != other.pstr_[i])
        {
            if (this->pstr_[i] - other.pstr_[i] > 0)
                return 1;
            else
                return -1;
        }
    }

    return 0;
}

bool MyString::operator>(const MyString& other) const
{
    return this->compare(other) == 1;
}
bool MyString::operator<(const MyString& other) const
{
    return this->compare(other) == -1;
}
bool MyString::operator>=(const MyString& other) const
{
    return this->compare(other) != -1;
}
bool MyString::operator<=(const MyString& other) const
{
    return this->compare(other) != 1;
}

```

```

}
bool MyString::operator!=(const MyString& other) const
{
    return this->compare(other) != 0;
}
bool MyString::operator==(const MyString& other) const
{
    return this->compare(other) == 0;
}

std::basic_ifstream<char>& operator>>(std::basic_ifstream<char>& is,
                                     MyString& str)
{
    str = "";

    char ch;

    while (is.get(ch))
    {
        if (!std::isspace(static_cast<unsigned char>(ch)))
        {
            str.append(1, ch);
            break;
        }
    }

    while (is.get(ch))
    {
        if (std::isspace(static_cast<unsigned char>(ch)))
        {
            is.unget();
            break;
        }

        str.append(1, ch);
    }

    return is;
}

std::basic_ofstream<char>& operator<<(std::basic_ofstream<char>& os,
                                     const MyString& str)
{
    os.write(str.c_str(), str.size());
    return os;
}

int MyString::find(std::string_view source_str) const
{
    return find(source_str, 0);
}

```

```

}

int MyString::find(std::string_view source_str, int index) const
{
    index = check_index(index, len_, 0, "find with index");

    int m = static_cast<int>(source_str.size());

    if (m == 0)
        return 0;

    AhoCorasickAutomaton automaton(source_str.data(), m);

    for (int i = index; i < len_; ++i)
    {
        automaton.step(static_cast<unsigned char>(pstr_[i]));

        if (automaton.is_match())
            return i - m + 1;
    }

    return -1;
}

char MyString::at(int index)
{
    if (index == 0 && len_ == 0)
        throw std::invalid_argument("Call method \"at\" at empty string");
    index = check_index(index, len_, 0, "at");

    return pstr_[index];
}

char MyString::at(iterator it)
{
    int index = it - begin();
    return at(index);
}

int MyString::to_int() { return static_cast<int>(this->to_float()); }

float MyString::to_float()
{
    for (int i = 0; i < this->len_; i++)
    {
        char tmp = this->pstr_[i];
        if ((tmp < '0' || tmp > '9') &&
            (tmp != '.' && tmp != '-' && tmp != '+' && tmp != 'f'))
        {
            std::string error_msg = "Cant transform this string to number ";

```

```

        error_msg += tmp;
        throw WrongTransformException(error_msg);
    }
}

float result = 0.0f;
float sign = 1.0f;
int i = 0;

if (this->pstr_[i] == '-')
{
    sign = -1.0f;
    i++;
}
else if (this->pstr_[i] == '+')
{
    i++;
}

while (i < this->len_ && this->pstr_[i] >= '0' && this->pstr_[i] <= '9')
{
    result = result * 10.0f + (this->pstr_[i] - '0');
    i++;
}

if (i < this->len_ && this->pstr_[i] == '.')
{
    i++;
    float weight = 0.1f;

    while (i < this->len_ && this->pstr_[i] >= '0' && this->pstr_[i] <= '9')
    {
        result += (this->pstr_[i] - '0') * weight;
        weight /= 10.0f;
        i++;
    }
}

return result * sign;
}
MyString::operator std::string_view() const
{
    return std::string_view(pstr_, len_);
}
my_string.h
#ifndef _MY_STRING_H_
#define _MY_STRING_H_

#include <exception>

```

```

#include <string_view>
#include <cstring>
#include <cstdlib>
#include <fstream>
#include <stdexcept>

#define DEBUG 1

class MyString
{
public:
    class WrongTransformException : public std::exception
    {
    public:
        WrongTransformException(std::string msg) : msg(msg) {}
        std::string what() { return msg; }

    private:
        std::string msg = "Wrong transformation";
    };

    using iterator = char*;
    using const_iterator = const char*;

    class reverse_iterator
    {
    public:
        explicit reverse_iterator(char* ptr) : ptr_it(ptr) {};

        char& operator*() const { return *ptr_it; }
        char* operator->() const { return ptr_it; }

        reverse_iterator& operator++()
        {
            --ptr_it;
            return *this;
        }
        reverse_iterator operator++(int)
        {
            reverse_iterator tmp = *this;
            --ptr_it;
            return tmp;
        }

        bool operator==(const reverse_iterator& other) const
        {
            return ptr_it == other.ptr_it;
        }
        bool operator!=(const reverse_iterator& other) const
        {

```

```

        return ptr_it != other.ptr_it;
    }

protected:
    char* ptr_it;
};

class const_reverse_iterator : public reverse_iterator
{
public:
    explicit const_reverse_iterator(const char* ptr)
        : reverse_iterator(const_cast<char*>(ptr))
    {
    }

    const char& operator*() const { return *ptr_it; }
    const char* operator->() const { return ptr_it; }

    const_reverse_iterator& operator++()
    {
        --ptr_it;
        return *this;
    }
    const_reverse_iterator operator++(int)
    {
        const_reverse_iterator tmp = *this;
        --ptr_it;
        return tmp;
    }
};

iterator begin();
iterator end();
const_iterator cbegin() const;
const_iterator cend() const;
reverse_iterator rbegin();
reverse_iterator rend();
const_reverse_iterator rcbegin() const;
const_reverse_iterator rcend() const;

// Constructors
MyString();
MyString(std::string_view source_str);
MyString(std::string_view source_str, int count);
MyString(int count, char ch);

MyString(MyString&& other); // Move constructor
MyString(int32_t number);
MyString(float number);

```

```

MyString(const char* source_str);
MyString(std::string source_str);
MyString(const MyString& other);

~MyString();

// clear str content
void clear();

// Clear not used ram
void shrink_to_fit();

// Equal operators
void operator=(std::string_view source_str);
void operator=(const char* source_str);
void operator=(const std::string& source_str);
void operator=(char ch);

MyString& operator=(MyString&& other);

MyString& operator=(const MyString& other);

// Gettes
const char* c_str() const;
int size() const;
int capacity() const;
bool empty() const;

// Insert by index
void insert(int index, int count, char ch);
void insert(int index, std::string_view source_str);
void insert(int index, std::string_view source_str, int count);
void insert(int index, std::string_view source_str, int s_index, int count);

void insert(iterator it, int count, char ch);
void insert(iterator it, std::string_view source_str);
void insert(iterator it, std::string_view source_str, int count);
void insert(iterator it, std::string_view source_str, int s_index,
            int count);

// Append to end of string
void append(int count, char ch);
void append(std::string_view source_str);
void append(std::string_view source_str, int count);
void append(std::string_view source_str, int s_index, int count);

// Delete sub string
void erase(int index, int count);

void erase(iterator it, int count);

```

```

// Replace sub string
void replace(int index, int count, std::string_view source_str);
void replace(int index, int count, std::string_view source_str,
             int s_count);
void replace(int index, int count, std::string_view source_str, int s_index,
             int s_count);

void replace(iterator it, int count, std::string_view source_str);
void replace(iterator it, int count, std::string_view source_str,
             int s_count);
void replace(iterator it, int count, std::string_view source_str,
             int s_index, int s_count);

// Get sub string
MyString substr(int index) const;
MyString substr(int index, int count) const;

MyString substr(const_iterator it) const;
MyString substr(const_iterator it, int count) const;

// Sum operators
MyString operator+(std::string_view source_str) const;
MyString& operator+=(std::string_view source_str);

// Index operator
char& operator[](int index);
const char& operator[](int index) const;

// Operator and methods to compare strings
short compare(const MyString& other) const;
bool operator>(const MyString& other) const;
bool operator<(const MyString& other) const;
bool operator>=(const MyString& other) const;
bool operator<=(const MyString& other) const;
bool operator!=(const MyString& other) const;
bool operator==(const MyString& other) const;

friend std::basic_ofstream<char>& operator<<(std::basic_ofstream<char>& os,
                                           const MyString& str);

friend std::basic_ifstream<char>& operator>>(std::basic_ifstream<char>& is,
                                           MyString& str);

// Finders
int find(std::string_view source_str) const;
int find(std::string_view source_str, int index) const;

char at(int index);

```

```

    char at(iterator it);

    int to_int();
    float to_float();

    operator std::string_view() const;

#ifdef DEBUG
    void pstr();
#endif

private:
    char* pstr_ = nullptr;
    int len_;
    int capacity_;

    void init(std::string_view sv);
    char* create_copy_of_pstr(int new_size) const;
    void delete_pstr_change_params(char* new_pstr, int new_len,
                                    int new_capacity);
    void my_insert(int index, int count, const char* data);
    int check_index(int index, int capacity, int count,
                    std::string error_msg) const;
};

#endif // _MY_STRING_H_

```

Приложение Б

MyString_wrapper.cpp

```

#include <pybind11/pybind11.h>

#include <stdexcept>
#include <string>
#include <string_view>

#include IMYSTRING

namespace py = pybind11;

namespace
{

std::string extract_string(const py::object& obj)
{
    if (py::isinstance<py::str>(obj))
        return obj.cast<std::string>();
}
}

```

```

    if (py::isinstance<py::bytes>(obj))
        return obj.cast<std::string>();

    if (py::isinstance<MyString>(obj))
    {
        MyString& ms = obj.cast<MyString&>();
        return std::string(ms.c_str(), static_cast<size_t>(ms.size()));
    }

    throw py::type_error("expected str, bytes or MyString");
}

bool looks_like_string_source(const py::object& obj)
{
    return py::isinstance<py::str>(obj) || py::isinstance<py::bytes>(obj) ||
        py::isinstance<MyString>(obj);
}

char extract_char(const py::object& obj)
{
    if (py::isinstance<py::int_>(obj))
        return static_cast<char>(obj.cast<int>());

    if (looks_like_string_source(obj))
    {
        std::string tmp = extract_string(obj);
        if (tmp.size() != 1)
            throw py::type_error(
                "expected a single character (str of length 1, bytes of "
                "length 1, or an int ordinal)");
        return tmp[0];
    }

    throw py::type_error(
        "expected a single character (str of length 1, bytes of length 1, "
        "or an int ordinal)");
}

void assign_string(MyString& self, const py::object& value)
{
    std::string tmp = extract_string(value);
    self = std::string_view(tmp);
}

void assign_char(MyString& self, const py::object& value)
{
    std::string tmp = extract_string(value);
    if (tmp.size() != 1)
        throw std::invalid_argument(
            "assign(char) expects exactly one character");
}

```

```

    self = tmp[0];
}

void assign_my_string(MyString& self, const MyString& value) { self = value; }

char getitem(MyString& self, int index) { return self[index]; }
void setitem(MyString& self, int index, char ch) { self[index] = ch; }
std::string to_py_str(const MyString& self)
{
    const char* p = self.c_str();
    return p ? std::string(p) : std::string();
}

} // namespace

PYBIND11_MODULE(mystring, m)
{
    py::class_<MyString> cls(m, "MyString");

    cls
        .def(py::init<>())
        .def(py::init(
            [](py::object a, py::object b)
            {
                if (a.is_none())
                    throw py::type_error(
                        "MyString() requires at least one argument");

                if (looks_like_string_source(a))
                {
                    std::string tmp = extract_string(a);
                    if (b.is_none())
                        return new MyString(std::string_view(tmp));
                    return new MyString(std::string_view(tmp),
                                         b.cast<int>());
                }

                if (py::isinstance<py::int_>(a) && !b.is_none() &&
                    !py::isinstance<py::float_>(b))
                {
                    try
                    {
                        char ch = extract_char(b);
                        return new MyString(a.cast<int>(), ch);
                    }
                    catch (const py::type_error&)
                    {
                    }
                }
            }
        ));
}

```

```

        if (b.is_none())
        {
            if (py::isinstance<py::float_>(a))
                return new MyString(a.cast<float>());
            if (py::isinstance<py::int_>(a))
                return new MyString(
                    static_cast<int32_t>(a.cast<int>()));
        }

        throw py::type_error(
            "unsupported MyString(...) argument combination");
    }),
    py::arg("a"), py::arg("b") = py::none())
.def(py::init<const MyString&>())

// --- misc ---
.def("clear", &MyString::clear)
.def("shrink_to_fit", &MyString::shrink_to_fit)

// --- getters ---
.def("c_str",
    [](&const MyString& self)
    {
        const char* p = self.c_str();
        return p ? std::string(p) : std::string();
    })
.def("size", &MyString::size)
.def("capacity", &MyString::capacity)
.def("empty", &MyString::empty)

// --- insert ---
.def(
    "insert", [](&MyString& self, int pos, int count, py::object ch)
    { self.insert(pos, count, extract_char(ch)); }, py::arg("pos"),
    py::arg("count"), py::arg("ch"))
.def(
    "insert",
    [](&MyString& self, int pos, py::object value, py::object count,
        py::object offset)
    {
        std::string tmp = extract_string(value);
        if (count.is_none())
            self.insert(pos, std::string_view(tmp));
        else if (offset.is_none())
            self.insert(pos, std::string_view(tmp), count.cast<int>());
        else
            self.insert(pos, std::string_view(tmp), count.cast<int>(),
                offset.cast<int>());
    },

```

```

py::arg("pos"), py::arg("value"), py::arg("count") = py::none(),
py::arg("offset") = py::none())

// --- append ---
.def(
    "append", [](MyString& self, int count, py::object ch)
    { self.append(count, extract_char(ch)); }, py::arg("count"),
    py::arg("ch"))
.def(
    "append",
    [](MyString& self, py::object value, py::object count,
        py::object offset)
    {
        std::string tmp = extract_string(value);
        if (count.is_none())
            self.append(std::string_view(tmp));
        else if (offset.is_none())
            self.append(std::string_view(tmp), count.cast<int>());
        else
            self.append(std::string_view(tmp), count.cast<int>(),
                offset.cast<int>());
    },
    py::arg("value"), py::arg("count") = py::none(),
    py::arg("offset") = py::none())

.def("erase",
    static_cast<void (MyString::*)(int, int)>(&MyString::erase))

// --- replace ---
.def(
    "replace",
    [](MyString& self, int pos, int len, py::object value,
        py::object count, py::object offset)
    {
        std::string tmp = extract_string(value);
        if (count.is_none())
            self.replace(pos, len, std::string_view(tmp));
        else if (offset.is_none())
            self.replace(pos, len, std::string_view(tmp),
                count.cast<int>());
        else
            self.replace(pos, len, std::string_view(tmp),
                count.cast<int>(), offset.cast<int>());
    },
    py::arg("pos"), py::arg("len"), py::arg("value"),
    py::arg("count") = py::none(), py::arg("offset") = py::none())

// --- substr ---
.def(
    "substr",

```

```

[] (const MyString& self, int pos, py::object len)
{
    if (len.is_none())
        return self.substr(pos);
    return self.substr(pos, len.cast<int>());
},
py::arg("pos"), py::arg("len") = py::none())

// --- find ---
.def(
    "find",
    [] (const MyString& self, py::object value, py::object from)
    {
        std::string tmp = extract_string(value);
        if (from.is_none())
            return self.find(std::string_view(tmp));
        return self.find(std::string_view(tmp), from.cast<int>());
    },
    py::arg("value"), py::arg("from") = py::none())

.def("at", static_cast<char (MyString::*)(int)>(&MyString::at))
.def("to_int", &MyString::to_int)
.def("to_float", &MyString::to_float)

// --- assign ---
.def("assign", &assign_string)
.def("assign_char", &assign_char)
.def("assign", &assign_my_string)

// --- operators ---
.def("__add__",
    [] (const MyString& self, py::object other)
    {
        std::string tmp = extract_string(other);
        return self + std::string_view(tmp);
    })
.def(
    "__iadd__",
    [] (MyString& self, py::object other) -> MyString&
    {
        std::string tmp = extract_string(other);
        self += std::string_view(tmp);
        return self;
    },
    py::is_operator())
.def("compare", &MyString::compare)
.def("__eq__", &MyString::operator==)
.def("__ne__", &MyString::operator!=)
.def("__lt__", &MyString::operator<)
.def("__gt__", &MyString::operator>)

```

```
.def("__le__", &MyString::operator<=)
.def("__ge__", &MyString::operator>=)

.def("__getitem__", &getitem)
.def("__setitem__", &setitem)

.def("__len__", &MyString::size)
.def("__str__", &to_py_str)
.def("__repr__", &to_py_str)

#ifdef DEBUG
    .def("pstr", &MyString::pstr)
#endif
;
}
```